SECURITY AUDIT OF

OP_NET Virtual Machine

Public Report

Version: 1.2

March 13, 2026

Verichains Lab

info@verichains.io

https://www.verichains.io

EXECUTIVE SUMMARY

This Security Audit Report was prepared by Verichains Lab on **March 13, 2026**. We would like to thank the OP_NET Team for trusting Verichains Lab in auditing smart contracts. Delivering high-quality audits is always our top priority.

This audit focused on identifying security flaws in code and the design of the OP_NET Virtual Machine. The scope of the audit is limited to the source code files provided to Verichains. Verichains Lab completed the assessment using manual, static, and dynamic analysis techniques.

During the audit and penetration testing process, the audit team had identified some issues in the source code.

TABLE OF CONTENTS

1. MANAGEMENT SUMMARY 4
1.1. About OP_NET Virtual Machine 4
1.2. Audit Methodology 4
1.3. Disclaimer 4
1.4. Acceptance Minute 5
2. AUDIT OVERVIEW 6
2.1. Audit Scope 6
2.2. Overview 6
3. FINDINGS 9
3.1. `C-01` - Groth16 Trusted Setup is Insecure Due to Deterministic Key Generation `[CRITICAL]` `[FIXED]` 9
3.2. `H-01` - Accumulate memory leaks over time `[HIGH]` `[FIXED]` 11
3.3. `H-02` - Futex Cleanup Race Condition `[HIGH]` `[FIXED]` 12
3.4. `H-03` - Incorrect memory padding during write `[HIGH]` `[FIXED]` 13
3.5. `H-04` - `deterministic_delay()` is not truly deterministic `[HIGH]` `[FIXED]` 15
3.6. `H-05` - High-S ECDSA signatures not rejected (signature malleability) `[HIGH]` `[FIXED]` 16
3.7. `L-01` - Memory Not Validated After Adjustment `[LOW]` `[ACKED]` 19
3.8. `L-02` - `RuntimePool` ignores pre-initialized runtimes `[LOW]` `[FIXED]` 20
3.9. `I-01` - Missing EIP-155 Replay-Protected `v` Value Support `[INFO]` `[OPEN]` 21
3.10. `I-02` - No `ecrecover` Address Recovery Method Available `[INFO]` `[OPEN]` 23
4. VERSION HISTORY 25

1. MANAGEMENT SUMMARY

1.1. About OP_NET Virtual Machine

OP_NET Virtual Machine is a virtual machine designed for evaluating smart contracts on the Bitcoin network using Rust and Wasmer. This VM is part of the broader OP_NET initiative, which aims to bring smart contract functionality to Bitcoin.

1.2. Audit Methodology

Our security audit process includes two steps:

- Static Analysis: automated security scanning using Rust security tooling.
- Manual Code Review: the codebase are manually analyzed to look for any potential problems.

Following is the list of commonly known vulnerabilities and security considerations that were examined during the audit:

- Memory Safety Issues
- Integer Safety
- Concurrency Issues
- Denial of Service (DoS)
- Cryptographic Vulnerabilities
- Unsafe Code Blocks
- FFI Boundary Issues
- Input Validation
- Logic Flaws
- Access Control

For vulnerabilities, we categorize the findings into categories as listed in table below, depending on their severity level:

Severity Level	Description
CRITICAL	A vulnerability that can disrupt the system functioning; creates a critical risk to the system; required to be fixed immediately.
HIGH	A vulnerability that could affect the desired outcome of executing with high impact; needs to be fixed with high priority.
MEDIUM	A vulnerability that could affect the desired outcome of executing with medium impact in a specific scenario; needs to be fixed.
LOW	An issue that does not have a significant impact, can be considered as less important.

1.3. Disclaimer

OP_NET Team acknowledges that the security services provided by Verichains, are conducted to the best of their professional abilities but cannot guarantee 100% coverage of all security vulnerabilities. OP_NET Team understands and accepts that despite rigorous auditing, certain vulnerabilities may remain undetected. Therefore, OP_NET Team agrees that Verichains shall not be held responsible or liable, and shall not be charged for any hacking incidents that occur due to security vulnerabilities not identified during the audit process.

1.4. Acceptance Minute

This final report served by Verichains to the OP_NET Team will be considered an Acceptance Minute. Within **seven** days, if no any further responses or reports is received from the OP_NET Team, the final report will be considered fully accepted by the OP_NET Team without the signature.

2. AUDIT OVERVIEW

2.1. Audit Scope

The audit focus on identifying security flaws in the code of OP_NET Virtual Machine. It was first conducted on commit `bd7c9b7c1728d4936bf49b8486b8377f53a7f466` from git repository `https://github.com/btc-vision/op-vm`. Latest audited commit is `6caac98834ff65c70b80af691dd0727c5066683c`.

This table summarizes the report revision history, mapping each report version to its corresponding git commit hash/PR.

Report Version	Git Commit/PR
0.1	`bd7c9b7c1728d4936bf49b8486b8377f53a7f466`
1.0	`ad1fb51e12e0985819b74f0fd30639e4f08ae72a`
1.1	`6139bd9103aff6df864263b39b200918aea39069`
1.2	`6caac98834ff65c70b80af691dd0727c5066683c`

2.2. Overview

This repository provides a **native Node.js module** (Neon) that embeds a **metered WebAssembly VM** (WasmerRunner) to execute smart-contract-like Wasm programs in a controlled, deterministic environment. Contracts interact with the outside world via a **host interface** implemented by the embedding application (JS callbacks).

2.2.1. Generic design

- **Execution model**: contracts are WebAssembly modules; execution is bounded by gas metering and resource limits.
- **Host integration**: contracts call imported host functions for state/IO/crypto; the embedding application supplies implementations (JS callbacks).
- **Separation of concerns**:
  - JS owns integration/orchestration (state backend, networking, persistence, indexing).
  - Rust owns VM safety, determinism controls, metering, memory/stack limits, and Wasm instantiation/execution.

2.2.2. Components

- **Node.js loader (`index.js`)**
  - Locates and loads the native binary (`index.node` / prebuilds) and exports the JS entrypoints.
- **Neon bindings (`src/lib.rs`, `src/interfaces/napi/`)**
  - Exposes the JS-facing `ContractManager`.
  - Bridges JS async callbacks into Rust.
  - Executes contract entrypoints off the main JS thread.
- **Contract runner (`src/domain/runner/`)**
  - Creates the execution environment (`CustomEnv`), instantiates Wasm modules, and calls contract exports.
  - Implements the host import surface used by contracts (module `env`).
- **VM internals (`src/domain/vm/`)**
  - Metering middleware and opcode cost model.

  - Memory/stack limiting tunables and hardened Wasm feature configuration. Optional feature modules (threading/atomics, table metering, proof/VDF features).

- **Contract Wasm**
  - The executed artifact; typically exports a small set of entrypoints (e.g. `start`, `onDeploy`, `execute`) and uses host imports.

2.2.3. Features

OP-VM is built for **deterministic execution**. It prohibits floating-point and treats SIMD-heavy/unsupported instruction paths as unusable, while keeping the enabled WebAssembly feature set conservative; some capabilities are only available behind Cargo feature flags. To reduce DoS risk, bulk operations are length-metered and clamped, and the runtime enforces gas/memory limits with tighter rules during initialization.

To keep execution **bounded**, the VM uses **gas metering**. Wasm is compiled with a metering middleware that maintains a remaining gas counter and decrements it as the contract runs; opcode costs are not uniform, so control flow, indirect calls, and memory-related work cost more than simple arithmetic. Host imports may also charge gas for expensive external actions (like storage, emits, or crypto helpers).

The VM enforces **resource limits** such as bounded memory growth and stack sizing. Some behavior is controlled at runtime (per-instance gas budget, debug mode, network selection, environment context), while other thresholds are compile-time choices (feature flags and hard-coded constants like max memory/pages and clamp limits).

Contracts interact with the outside world through a **pluggable host interface** implemented by JS callbacks. Rust bridges these calls through a small encoding/decoding layer: requests are turned into JS arguments (via `AsArguments`) and Promise results are decoded back into Rust (via `FromJsObject`), then written into Wasm memory or used to update execution state.

Finally, OP-VM provides **instance management** from Node.js: reserve ids, instantiate contracts, set execution context, run entrypoints, inspect results (exit data/gas), and clean up instances/caches. The contract-visible surface is a compact set of host imports for control, context, state/system calls, events/IO, and crypto/validation.

2.2.4. Execution flow

1. **JavaScript: imports** the module.
2. **JavaScript: instantiates** a contract from Wasm bytecode.
3. **JavaScript: sets** the execution environment (block/tx context, addresses, flags, etc.).
4. **JavaScript: calls** the contract's entrypoint (deploy/execute).
5. **Rust: spawns** a thread to run the Wasm runner.
6. **Wasm: calls** host imports when it needs external data; **Rust** forwards to **JavaScript** callbacks and resumes when they resolve.
7. **Rust: returns** an execution result (status + return data + accounting metadata) back to **JavaScript**.

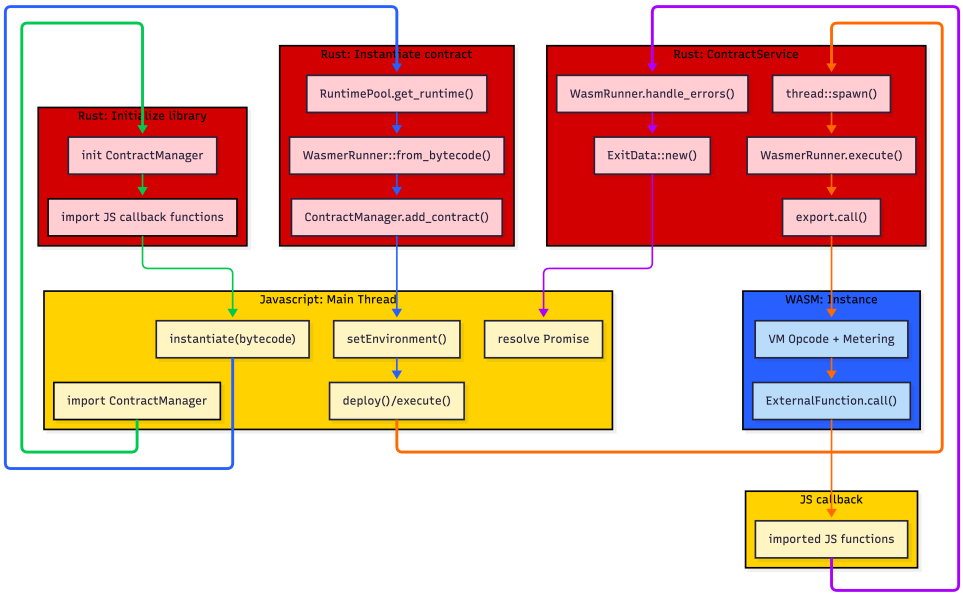


Figure 1: Execution Flow

3. FINDINGS

3.1. `C-01` - Groth16 Trusted Setup is Insecure Due to Deterministic Key Generation `[CRITICAL]` `FIXED`

SCOPE

- **Commit**: `bd7c9b7c1728d4936bf49b8486b8377f53a7f466`
- **Files**: `src/domain/vm/vdf/snark.rs`

DESCRIPTION

The current ZK-SNARK implementation in `src/domain/vm/vdf/snark.rs` contains a critical security vulnerability that compromises the soundness of the entire system. The Groth16 trusted setup process uses a deterministic, hardcoded seed, allowing any attacker to forge proofs for invalid statements.

This affects any component that relies on the `prove` and `verify` functions, including the `atomic.wait` VDF-based timeout mechanism.

The Vulnerability

The Groth16 proving system requires a secure, one-time trusted setup to generate a `ProvingKey` (PK) and `VerifyingKey` (VK). This process involves generating secret random values (often called "toxic waste") that must be destroyed. If an attacker knows these values, they can break the **soundness** of the SNARK, meaning they can create valid-looking proofs for false statements.

The current implementation in `gen_keys` uses a deterministic Random Number Generator (RNG) seeded with a public constant:

```
// src/domain/vm/vdf/snark.rs#L109-L120
fn gen_keys(t: u64) -> Result<(ProvingKey<Bls12_381>, VerifyingKey<Bls12_381>)> {
    let dummy = PoseidonIterCircuit {
        init: Fr::zero(),
        iterations: t,
        pub_hash_fr: Fr::zero(),
    };
    let mut rng = FixedChaCha::seed_from_u64(42);
    let pk = Groth16::<Bls12_381>::generate_random_parameters_with_reduction(dummy, &mut rng)
        .context("Groth16 parameter generation failed")?;
    Ok((pk.clone(), pk.vk))
}
```

```
fn gen_keys(t: u64) -> Result<(ProvingKey<Bls12_381>, VerifyingKey<Bls12_381>)> {
    let dummy = PoseidonIterCircuit {
        init: Fr::zero(),
        iterations: t,
        pub_hash_fr: Fr::zero(),
    };

    let mut rng = FixedChaCha::seed_from_u64(42);
    let pk = Groth16::<Bls12_381>::generate_random_parameters_with_reduction(dummy, &mut rng)
        .context("Groth16 parameter generation failed")?;
    Ok((pk.clone(), pk.vk))
}
```

Because the seed (`42`) is public, any attacker can re-run this function to reconstruct the exact "toxic waste" and generate the same `ProvingKey` .

Impact

1. **Proof Forgery:** An attacker with the `ProvingKey` can forge a valid proof for any statement. In the context of our VDF, they can create a proof that they performed `t` iterations of the VDF without actually doing the computational work.
2. **Authorization Bypass:** This breaks the `atomic.wait` security model. An attacker can forge a proof to bypass the "proof-of-work" check, allowing them to call the `wait` function without paying the computational cost. This could be used to mount a Denial-of-Service (DoS) attack.
3. **Untrustworthy Proving Service:** Any proofs generated by the host's "proving service" are worthless, as they are derived from the same insecure setup and can be forged by anyone.

Additionally, the `prove` function is called for every proof generation, which is conceptually incorrect and extremely inefficient. The trusted setup should be a one-time, offline process for a given circuit (i.e., for a given VDF difficulty `t`).

RECOMMENDATION

1. **Create a Secure, Offline Trusted Setup Script:**
   - Create a separate binary/tool for key generation. This tool must not be part of the runtime library.
   - This tool **must** use a cryptographically secure RNG from the operating system, such as `rand::thread_rng()` .
   - For each supported VDF difficulty `t` , run this tool once to generate a `proving_key.bin` and `verifying_key.bin` . These keys are the trusted parameters for the system.
2. **Refactor the Runtime Library:**
   - Remove the `gen_keys` function entirely from the runtime library.
   - The host environment should load the pre-generated, trusted `ProvingKey` and `VerifyingKey` files at startup (e.g., into a `HashMap<u64, VerifyingKey>`).
   - Modify the `prove` function to accept a `&ProvingKey` as an argument instead of generating one on the fly.
   - Modify the `verify` function to **always** use the host's trusted `VerifyingKey` from the map, ignoring any `vk` provided by the guest.
3. **Fix Proof Generation Randomness:**
   - The `prove` function currently uses another deterministic seed (`FixedChaCha::seed_from_u64(99)`) to create the proof. This breaks the zero-knowledge property. This should also be replaced with `rand::thread_rng()` .

UPDATES

The feature has been removed in commit `9fa3fa547c6e377e1048c4542d30e45e8812a7fc` .

3.2. H-01 - Accumulate memory leaks over time [HIGH] FIXED

SCOPE

- **Commit:** bd7c9b7c1728d4936bf49b8486b8377f53a7f466
- **Files:** src/domain/vm/atomic/atomic_shim.rs

DESCRIPTION

```
let vk = if vk_bytes.is_empty() {
    return Ok(NOT_AUTHORIZED);
} else {
    match deserialize_vk(&vk_bytes) {
        Some(v) => Box::leak(Box::new(v)), // 'static lifetime hack
        None => return Ok(NOT_AUTHORIZED),
    }
};
```

The `wait_async` function permanently leaks memory every time it's called with valid `vk_bytes` . The code uses `Box::leak()` as a `'static` lifetime hack, but this memory is never reclaimed.

Impact per call:

1. Deserialize a `VerifyingKey<Bls12_381>`
2. Box the key in heap memory
3. Permanently leak that boxed memory using `Box::leak()`
4. Memory persists for the entire process lifetime

In long-running services or smart contract environments, this creates unbounded memory growth with no upper limit. Each leaked key consumes memory that accumulates over thousands or millions of function calls, potentially leading to memory exhaustion and performance degradation.

RECOMMENDATION

Replace the memory leak with proper memory management using one of these approaches:

Option 1: Key Caching

```
use std::sync::OnceLock;
use std::collections::HashMap;
static VK_CACHE: OnceLock<Mutex<HashMap<Vec<u8>, Arc<VerifyingKey<Bls12_381>>>>> = OnceLock::new();
let vk = Arc::new(deserialize_vk(&vk_bytes)?);
```

Option 2: Refactor verify function to accept borrowed keys instead of requiring `'static` lifetime, eliminating the need for the lifetime hack entirely.

UPDATES

This issue was fixed in commit `844faf062b903f2090b095c1067b7f5d9e1e34f4` : the `Box::leak()` function is completely removed.

3.3. H-02 - Futex Cleanup Race Condition [HIGH] FIXED

SCOPE

- **Commit:** bd7c9b7c1728d4936bf49b8486b8377f53a7f466
- **Files:** src/domain/runner/instance_wrapper.rs , src/domain/vm/atomic/atomic_shim.rs

DESCRIPTION

```
impl Drop for InstanceWrapper {
    fn drop(&mut self) {
        if Arc::strong_count(&self.futexes) == 1 {
            self.futexes.retain(|_, q| Arc::strong_count(q) > 1);
        }
    }
}
```

The InstanceWrapper::Drop implementation lacks proper shutdown for futex wait queues. When the instance is dropped, tasks waiting on futexes hang indefinitely because:

1. The retain() logic uses Arc::strong_count to determine which queues to keep, but this reflects ownership, not active waiters
2. Tasks holding Arc<WaitQueue> references remain blocked in wait_for_change() with no wake mechanism
3. The entire DashMap gets dropped without notifying waiting tasks

```
// Reproduction scenario
let wait_queue = instance.futex_for(0x1000);
tokio::spawn(async move {
    wait_queue.wait_for_change(ticket).await; // Hangs forever after drop
});
drop(instance); // Tasks never wake up
```

This causes thread leaks, resource consumption, and potential deadlocks in WebAssembly applications using futex operations.

RECOMMENDATION

Add explicit shutdown mechanism to WaitQueue with closed state tracking and proper notification:

```
impl Drop for InstanceWrapper {
    fn drop(&mut self) {
        if Arc::strong_count(&self.futexes) == 1 {
            let queues: Vec<_> = self.futexes.iter().map(|e| e.value().clone()).collect();
            self.futexes.clear();
            for q in queues {
                q.shutdown(); // Wake all waiters + mark closed
            }
        }
    }
}
```

UPDATES

Fixed in commit ab6636869cbd9db402f1c4ef77b290dcd510dd95 .

3.4. H-03 - Incorrect memory padding during write [HIGH] FIXED

SCOPE

- **Commit**: bd7c9b7c1728d4936bf49b8486b8377f53a7f466
- **Files**: src/domain/runner/import_functions/common/data_slice_writer.rs

DESCRIPTION

```rust
impl DataSliceWriter {
    pub fn write_data_and_padding_to_memory(
        store: &mut StoreMut,
        instance: &InstanceWrapper,
        result_data: &[u8],
        offset: u32,
        length: u32,
        result_ptr: u32,
    ) -> Result<(), RuntimeError> {
        Self::write_data_slice_to_memory(
            store,
            &instance,
            result_data,
            offset,
            length,
            result_ptr,
        )?;
        Self::write_data_padding_to_memory(
            store,
            &instance,
            result_data,
            offset,
            length,
            result_ptr,
        )
    }

    fn write_data_slice_to_memory(
        store: &mut StoreMut,
        instance: &InstanceWrapper,
        result_data: &[u8],
        offset: u32,
        length: u32,
        result_ptr: u32,
    ) -> Result<(), RuntimeError> {
        let result_data_sliced =
            Self::slice_in_bounds(result_data, offset as usize, length as usize)
                .ok_or(RuntimeError::new("Error slicing data"))?;

        if !result_data_sliced.is_empty() {
            instance
                .write_memory(&store, result_ptr as u64, result_data_sliced)
                .map_err(|_e| RuntimeError::new("Error writing data to memory"))?;
        }
        Ok(())
    }

    fn slice_in_bounds(data: &[u8], offset: usize, length: usize) -> Option<&[u8]> {
        let start = min(offset, data.len());
        let end = min(offset + length, data.len());
        data.get(start..end)
    }

    fn write_data_padding_to_memory(
        store: &mut StoreMut,
        instance: &InstanceWrapper,
        result_data: &[u8],
        offset: u32,
        length: u32,
        result_ptr: u32,
    ) -> Result<(), RuntimeError> {
        if result_data.len() < (offset + length) as usize {
            let zero_bytes_length = (offset + length) as usize - result_data.len();
            let zero_bytes_vec = vec![0; zero_bytes_length];
            let ptr = result_ptr as u64 + result_data.len() as u64;
            instance
                .write_memory(&store, ptr, &zero_bytes_vec)
                .map_err(|_e| RuntimeError::new("Error writing data to memory"))?;
        }
        Ok(())
```

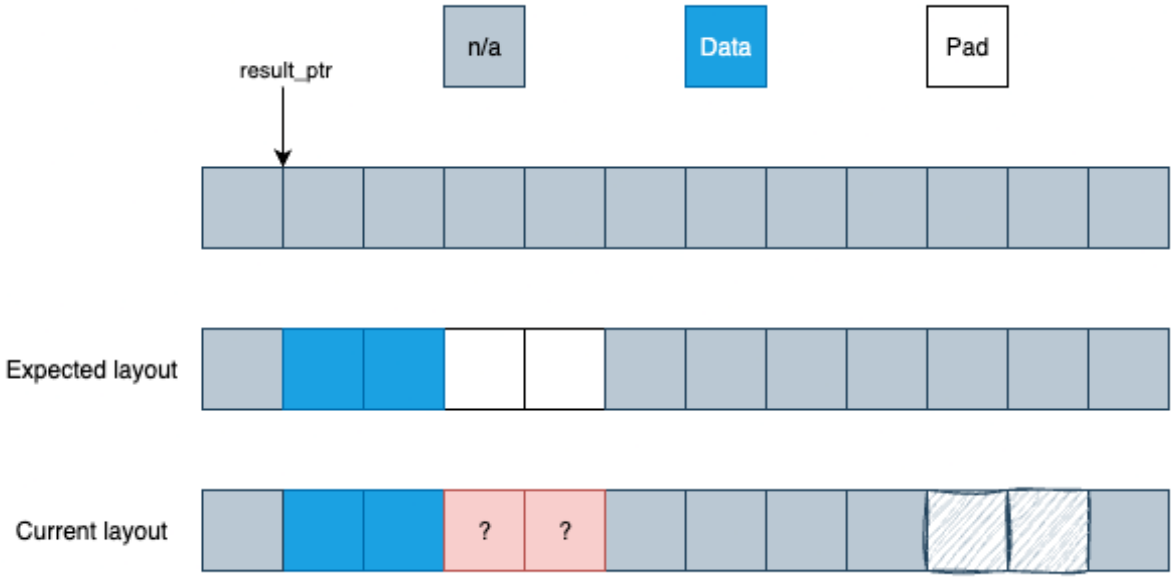
```
    }
}
```

The function `write_data_padding_to_memory()` does not take into account the actual amount of data written, as determined by `slice_in_bounds()` . Instead, it incorrectly assumes that the entire data block was written and applies padding based on that assumption.

This leads to two issues:

1. The designated padding cell might remain untouched, potentially still containing stale or tampered data.
2. The padding may overwrite adjacent data beyond the actual written size.

For example, consider `data.length = 8` , `offset = 6` , and `length = 4` . In the expected layout, the last 2 bytes of data are written to `ptr` , followed immediately by 2 bytes of padding. In the current (incorrect) layout, the last 2 bytes of data are written to `ptr` , but the remaining 2 bytes are left untouched. The 2-byte padding is then written further ahead in memory - potentially overwriting unrelated data.



UPDATES

Fixed in commit `8b7bff2c7ee1597085061644ada1ddbb413db050` .

3.5. H-04 - deterministic_delay() is not truly deterministic [HIGH] FIXED

SCOPE

- **Commit**: bd7c9b7c1728d4936bf49b8486b8377f53a7f466
- **Files**: src/domain/vm/atomic/atomic_shim.rs

DESCRIPTION

```
async fn deterministic_delay(seed: u64, diff: u64) -> Result<(), RuntimeError> {
    let mut vdf = VdfState::new(&little_endian_u64(seed))?;
    for n in 0..diff {
        prove_one_step(&mut vdf)?;

        if n & 0xFF == 0 {
            tokio::task::yield_now().await;
        }
    }

    Ok(())
}
```

The current implementation of deterministic_delay() relies on prove_one_step() and tokio::task::yield_now() to simulate a deterministic delay. However, according to the documentation of tokio::task::yield_now() :

> It is generally not guaranteed that the runtime behaves like you expect it to when deciding which task to schedule next after a call to yield_now() . In particular, the runtime may choose to poll the task that just ran yield_now() again immediately without polling any other tasks first. For example, the runtime will not drive the IO driver between every poll of a task, and this could result in the runtime polling the current task again immediately even if there is another task that could make progress if that other task is waiting for a notification from the IO driver.

> In general, changes to the order in which the runtime polls tasks is not considered a breaking change, and your program should be correct no matter which order the runtime polls your tasks in.

Therefore, relying on tokio::task::yield_now() to provide a deterministic delay appears to be impractical.

UPDATES

OP_NET team has acknowledged this issue. This function has been removed in commit dc4c8ff9b5a330eaae4dffa709e720311940c588 .

3.6. H-05 - High-S ECDSA signatures not rejected (signature malleability) [HIGH] FIXED

SCOPE

- Commit: `3143aa61c46d51fffea9169cde29af1269e2e8af`
- File: `op-vm/src/domain/runner/import_functions/verify_signatures_import.rs`

DESCRIPTION

Background

An ECDSA signature over the secp256k1 curve consists of the pair `(r, s)` where both components are elements of `Z_n` (integers modulo the curve order `n`). Due to the algebraic symmetry of the elliptic curve, for every valid signature `(r, s)` , the pair `(r, n - s)` is also a valid signature for the same message and public key. This property is known as **ECDSA signature malleability**.

A signature is classified as:

- **Low-S** if `s ≤ n/2`
- **High-S** if `s > n/2`

Both Bitcoin and Ethereum enforce low-S requirements to obtain canonical signature forms and eliminate malleability vectors.

Bitcoin Standard: BIP-62 / BIP-146 - Reject High-S

Bitcoin Core enforces the `LOW_S` rule at the policy (standardness) layer, not the consensus layer. Under this rule, transactions where `s > n/2` in an ECDSA signature are rejected from the mempool and not relayed by standard nodes. However, if such a transaction were included in a mined block, it would be accepted as consensus-valid.

Ethereum Standard: EIP-2 (Homestead) - Reject High-S for Transactions

Ethereum enforced the low-S requirement in **EIP-2** (activated at the Homestead hard fork, block 1,150,000):

- All transaction signatures **MUST** have `s ≤ secp256k1n / 2` .
- Transaction signatures with `s > secp256k1n / 2` are considered **invalid**.
- The `ecrecover` precompile itself still accepts high-S signatures (for backward compatibility with legacy use cases such as verifying Bitcoin-originated signatures), but any application-level signature verification should enforce low-S to prevent malleability.

Current Implementation Analysis

```
fn verify_ecdsa_bitcoin(
    store: StoreMut,
    instance: InstanceWrapper,
    public_key_ptr: u32,
    signature_ptr: u32,
    message_ptr: u32,
) -> Result<u32, RuntimeError> {
    let sig_bytes: [u8; 64] = instance
        .read_memory(&store, signature_ptr as u64, 64)
        .map_err(|_e| RuntimeError::new("Error reading ECDSA signature from memory"))?
        .try_into()
        .map_err(|_e| RuntimeError::new("ECDSA signature must be exactly 64 bytes"))?;

    let message_hash: [u8; 32] = instance
        .read_memory(&store, message_ptr as u64, 32)
        .map_err(|_e| RuntimeError::new("Error reading ECDSA message hash from memory"))?
        .try_into()
        .map_err(|_e| RuntimeError::new("ECDSA message hash must be exactly 32 bytes"))?;
```

```
    let signature = match EcdsaSignature::from_slice(&sig_bytes) {
        Ok(sig) => sig,
        Err(_e) => {
            return Ok(0);
        },
    };
    // BIP-0062: enforce canonical low-S form. normalize_s() returns Self,
    // always valid. If S was already low it returns an identical copy.
    let signature = signature.normalize_s();
    let verifying_key = match Self::read_ecdsa_public_key(&store, &instance, public_key_ptr) {
        Ok(key) => key,
        Err(_e) => {
            return Ok(0);
        },
    };
    let result = verifying_key.verify_prehash(&message_hash, &signature);
    Ok(result.is_ok() as u32)
}
```

Issue: The function calls `signature.normalize_s()` which converts a high-S value to its low-S equivalent (`s' = n - s`) rather than rejecting it. While this ensures verification succeeds with the canonical form, it **silently accepts malleable signatures**. An attacker can submit a transaction with a high-S signature `(r, n - s)` , and the system will normalize it and verify it as valid. This deviates from the Bitcoin consensus rule (BIP-146) which **rejects** high-S signatures entirely.

Malleability impact: If an observer sees a valid transaction with signature `(r, s)` (where `s` is low), they can construct an alternative signature `(r, n - s)` and submit it. Because `normalize_s()` will convert `n - s` back to `s` , both signatures verify successfully. This enables:

- **Transaction ID mutation:** Two different transactions (with different serialized signatures) can map to the same logical operation, causing txid-dependent tracking systems to desynchronize.
- **Replay attacks in application-level logic:** Smart contracts that track "used signatures" by hashing the raw `(r, s)` bytes will see the malleated signature as a distinct, unused signature.
- **Nonce/permit replay:** In OP_20 token `permit()` flows, if the signature hash does not cover a normalized form, an attacker can replay a permit with the flipped `s` , potentially re-executing the approval.

3.6.2.4.1. `verify_ecdsa_ethereum` (lines 294–342)

```
fn verify_ecdsa_ethereum(
    store: StoreMut,
    instance: InstanceWrapper,
    public_key_ptr: u32,
    signature_ptr: u32,
    message_ptr: u32,
) -> Result<u32, RuntimeError> {
    let sig_bytes: [u8; 65] = instance
        .read_memory(&store, signature_ptr as u64, 65)
        .map_err(|_e| RuntimeError::new("Error reading ECDSA signature from memory"))?
        .try_into()
        .map_err(|_e| RuntimeError::new("ECDSA signature must be exactly 65 bytes"))?;
    let message_hash: [u8; 32] = instance
        .read_memory(&store, message_ptr as u64, 32)
        .map_err(|_e| RuntimeError::new("Error reading ECDSA message hash from memory"))?
        .try_into()
        .map_err(|_e| RuntimeError::new("ECDSA message hash must be exactly 32 bytes"))?;
    let signature = match EcdsaSignature::from_slice(&sig_bytes[..64]) {
        Ok(sig) => sig,
        Err(_) => return Ok(0),
    };
    let v_byte = sig_bytes[64];
    let recovery_id_value = match v_byte {
        0 | 1 => v_byte,
        27 | 28 => v_byte - 27,
        _ => return Ok(0),
```

```
    };
    let recovery_id = match RecoveryId::from_byte(recovery_id_value) {
        Some(id) => id,
        None => return Ok(0),
    };
    let recovered_key =
        match VerifyingKey::recover_from_prehash(&message_hash, &signature, recovery_id) {
            Ok(key) => key,
            Err(_) => return Ok(0),
        };
    let provided_key = match Self::read_ecdsa_public_key(&store, &instance, public_key_ptr) {
        Ok(key) => key,
        Err(_) => return Ok(0),
    };
    Ok((recovered_key == provided_key) as u32)
}
```

Issue: There is **no low-S check at all** in the Ethereum verification path. The function's inline comment (line 291–293) explicitly states:

> *"Low-S normalization is NOT applied before recovery because flipping S changes which key gets recovered, breaking valid pre-EIP-2 signatures."*

While this reasoning is technically correct for the `ecrecover` precompile itself, it is **insufficient for application-level verification** where the platform aims to provide Ethereum-compatible security guarantees. The code should **reject** (return `0`) if `s > n/2` , matching the behavior expected by EIP-2 for transaction-level signatures and by all modern Ethereum signature verification libraries (e.g., OpenZeppelin `ECDSA.tryRecover`).

Malleability impact (Ethereum path): Without any low-S enforcement, an attacker who observes a valid Ethereum-style signature `(r, s, v)` can compute `(r, n - s, v')` where `v' = v ^ 1` (flipped recovery parity). The `ecrecover` with the flipped values will recover the **same** address. This means:

- Two distinct 65-byte signature blobs both verify as valid for the same message and signer.
- Any smart contract that uses raw signature bytes as a uniqueness key (e.g., permit nonces, signature-gated operations) can be bypassed.

RECOMMENDATION

Both `verify_ecdsa_bitcoin` and `verify_ecdsa_ethereum` should **reject** high-S signatures instead of normalizing or ignoring the S value.

UPDATES

This issue was fixed in commit `6caac98834ff65c70b80af691dd0727c5066683c` .

3.7. L-01 - Memory Not Validated After Adjustment [LOW] ACKED

SCOPE

- **Commit**: bd7c9b7c1728d4936bf49b8486b8377f53a7f466
- **Files**: src/domain/vm/limiting_tunables.rs

DESCRIPTION

```
/// Takes an input memory type as requested by the guest and sets
/// a maximum if missing. The resulting memory type is final if
/// valid. However, this can produce invalid types, such that
/// validate_memory must be called before creating the memory.
fn adjust_memory(&self, requested: &MemoryType) -> MemoryType {
   let mut adjusted = requested.clone();
   if requested.maximum.is_none() {
       adjusted.maximum = Some(self.max_pages);
   }
   adjusted
}
```

```
fn memory_style(&self, memory: &MemoryType) -> MemoryStyle {
   let adjusted = self.adjust_memory(memory);
   self.base.memory_style(&adjusted)
}
```

According to the comment in `adjust_memory()` , it may produce invalid types; therefore, in `memory_style()` , `validate_memory()` must be called before passing the adjusted memory to the base.

UPDATES

In commit `74d3f43801629a4aaba1bbf2af41b4aae8d3dac8` , the comment was updated to clarify that `adjust_memory()` is intended for internal use only within `create_host_memory()` and `create_vm_memory()` , and should not be invoked elsewhere.

3.8. `L-02` - `RuntimePool` ignores pre-initialized runtimes `[LOW]` `FIXED`

SCOPE

- Commit: `bd7c9b7c1728d4936bf49b8486b8377f53a7f466`
- Files: `src/interfaces/napi/runtime_pool.rs`

DESCRIPTION

```rust
impl RuntimePool {
    pub fn new(size: usize) -> Self {
        let mut runtimes = VecDeque::with_capacity(size);
        for _ in 0..size {
            let runtime = Self::create_runtime();
            runtimes.push_back(runtime);
        }
        Self {
            runtimes: Mutex::new(runtimes),
            idling: Mutex::new(Arc::new(AtomicUsize::new(0))),
            expected_size: Mutex::new(Arc::new(AtomicUsize::new(size))),
        }
    }
}
```

The `new()` method initializes `RuntimePool` with `size` runtimes but sets `idling` to `0` , meaning no runtime is initially marked as available. This mismatch causes `RuntimePool` to ignore the pre-created runtimes and create new ones while `idling` remains `0` , leaving the pre-created runtimes unused until `return_runtime()` is called (which increments `idling`).

RECOMMENDATION

Correct initialize `idling` to `size` or remove the pre-created runtimes.

UPDATES

This issue was fixed in commit `8c78bda254cf3c28ec9a4ba941fdd3ac79c23946` .

3.9. `I-01` - Missing EIP-155 Replay-Protected `v` Value Support `[INFO]` `OPEN`

SCOPE

- Commit: `13143aa61c46d51fffea9169cde29af1269e2e8af1`
- File: `op-vm/src/domain/runner/import_functions/verify_signatures_import.rs`

DESCRIPTION

Background

EIP-155 (Simple Replay Attack Protection) introduced chain-ID-based replay protection for Ethereum transactions. Under EIP-155, the `v` value in a signature is encoded as:

```
v = recovery_id + 35 + 2 * chainID
```

Where:

- `recovery_id` ∈ {0, 1} - the y-parity of the recovered public key point
- `chainID` - the unique identifier for the target chain (e.g., 1 for Ethereum mainnet, 137 for Polygon)

The following code snippet from `go-ethereum` shows how the Ethereum handles the EIP-155

```go
func isProtectedV(V *big.Int) bool {
  if V.BitLen() <= 8 {
    v := V.Uint64()
    return v != 27 && v != 28 && v != 1 && v != 0
  }
  // anything not 27 or 28 is considered protected
  return true
}

func sanityCheckSignature(v *big.Int, r *big.Int, s *big.Int, maybeProtected bool) error {
  if isProtectedV(v) && !maybeProtected {
    return ErrUnexpectedProtection
  }

  var plainV byte
  if isProtectedV(v) {
    chainID := deriveChainId(v).Uint64()
    plainV = byte(v.Uint64() - 35 - 2*chainID)
  } else if maybeProtected {
    // Only EIP-155 signatures can be optionally protected. Since
    // we determined this v value is not protected, it must be a
    // raw 27 or 28.
    plainV = byte(v.Uint64() - 27)
  } else {
    // If the signature is not optionally protected, we assume it
    // must already be equal to the recovery id.
    plainV = byte(v.Uint64())
  }
  if !crypto.ValidateSignatureValues(plainV, r, s, false) {
    return ErrInvalidSig
  }

  return nil
}
```

Current Implementation Analysis

The current `v` byte parsing in `verify_ecdsa_ethereum` (lines 318–323):

```rust
let v_byte = sig_bytes[64];
let recovery_id_value = match v_byte {
    0 | 1 => v_byte,
    27 | 28 => v_byte - 27,
    _ => return Ok(0),
};
```

This implementation accepts only four `v` values:

`v` byte	Interpretation	Mapped `recovery_id`
`0`	Raw recovery ID	`0`
`1`	Raw recovery ID	`1`
`27`	Legacy pre-EIP-155	`0`
`28`	Legacy pre-EIP-155	`1`

Any other `v` value causes the function to return `0` (invalid). This means EIP-155 protected signatures with chain-specific `v` values (e.g., `35` , `36` for chain ID 1; `37` , `38` for chain ID 2, etc.) are unconditionally rejected.

RECOMMENDATION

Extend the `v` -byte parsing to support EIP-155 chain-ID-encoded values.

3.10. `I-02` - No `ecrecover` Address Recovery Method Available `[INFO]` `OPEN`

SCOPE

- Commit: `3143aa61c46d51fffea9169cde29af1269e2e8af`
- Files:
  - `btc-runtime/runtime/env/BlockchainEnvironment.ts`
  - `op-vm/src/domain/runner/import_functions/verify_signatures_import.rs`

DESCRIPTION

Background

In Ethereum, `ecrecover` is a precompile (at address `0x01`) that recovers the signer's address from a signature. Its critical design property is that **the caller does not need to provide the public key** - only the signature and message hash are required:

```
// Ethereum ecrecover: no public key input needed
address signer = ecrecover(hash, v, r, s);
require(signer == expectedAddress);
```

The precompile takes `(hash, v, r, s)` and **returns the 20-byte address** derived from the recovered public key. This is the fundamental primitive underlying signature-based authentication patterns across the Ethereum ecosystem (EIP-712 typed data, EIP-2612 permits, meta-transactions, account abstraction).

This design enables contracts to verify signatures using only the signer's address - a compact 20-byte identifier that is already the standard unit of identity in Ethereum. The signer's full public key never needs to be stored, transmitted, or known by the verifying contract.

Current Implementation Analysis

OP_NET's `verifyECDSASignature()` takes a fundamentally different approach - it requires the caller to provide the **full SEC1 public key** (33, 64, or 65 bytes) as an explicit input:

```
public verifyECDSASignature(
    publicKey: Uint8Array,   // ← full SEC1 public key REQUIRED
    signature: Uint8Array,   // 65 bytes: r(32) || s(32) || v(1)
    hash: Uint8Array,        // 32-byte message hash
): boolean
```

Internally, the host-side `verify_ecdsa_ethereum` does perform key recovery, but the recovered key is never returned - it is only compared against the caller-provided key.

There is **no method** that returns the recovered public key or its derived address to the caller.

Impact

1. **Cannot verify signatures without the public key:** Unlike Ethereum where a contract only needs the signer's address (20 bytes), OP_NET contracts must have access to and pass in the full public key (33–65 bytes). This forces contracts to either store full public keys on-chain or require callers to supply them alongside every signature, increasing both storage costs and calldata size.
2. **Reduced Ethereum compatibility:** Standard Ethereum patterns like OpenZeppelin's `ECDSA.recover(hash, signature)` return an address that the caller compares against the expected signer - no public key input required. Developers porting Ethereum contracts to OP_NET cannot use this established pattern and must redesign their signature verification logic to include public key management.

RECOMMENDATION

Consider adding an `ecrecover` -style host function that accepts only `(signature, hash)` and returns the recovered public key or derived address to the guest. This would:

- Allow signature verification **without requiring the public key as input**
- Enable Ethereum-compatible signature verification patterns

4. VERSION HISTORY

Version	Date	Status/Change	Changed by
0.1	*Aug 01, 2025*	Draft Report	Verichains Lab
1.0	*Jan 19, 2026*	Private Report	Verichains Lab
1.1	*Feb 02, 2026*	Private Report	Verichains Lab
1.2	*Mar 13, 2026*	Public Report	Verichains Lab